

AI Assisted Coding Lab 12.3

Sanjana Kolipaka

2303A52139

Batch – 41

Question 1: Sorting Student Records for Placement Drive

Prompt: Generate a python code for Sorting student records for placement drive. Each record should contain the student's name, roll number, and CGPA. Implement the quick sort and merge sort algorithms to sort the records based on CGPA. Include a function that stores student records in a list and demonstrates the sorting functionality with a real-life sample data. Provide clean and efficient code with proper comments explaining the logic. Measure and compare runtime performance for large datasets. Write a function to display the top 10 students based on their CGPA.

```
import time
import random
from typing import List, Dict

class Student:
    """Class to represent a student record"""
    def __init__(self, name: str, roll_number: str, cgpa: float):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"Student(name='{self.name}', roll_number='{self.roll_number}', cgpa={self.cgpa})"

def quick_sort(students: List[Student], low: int, high: int) -> None:
    """
    Quick Sort algorithm to sort students by CGPA in descending order
    Time Complexity: O(n log n) average, O(n^2) worst case
    Space Complexity: O(log n) for recursion stack

    Args:
        students: List of Student objects
        low: Starting index
        high: Ending index
    """
    if low < high:
        # Partition the array and get pivot index
        pivot_index = partition(students, low, high)

        # Recursively sort elements before and after partition
        quick_sort(students, low, pivot_index - 1)
        quick_sort(students, pivot_index + 1, high)
```

```
def partition(students: List[Student], low: int, high: int) -> int:
    """
    Partition function for Quick Sort
    Uses last element as pivot and arranges elements in descending order of CGPA

    Args:
        students: List of Student objects
        low: Starting index
        high: Ending index

    Returns:
        Final position of pivot element
    """
    pivot = students[high].cgpa # Choose last element as pivot
    i = low - 1 # Index of smaller element

    for j in range(low, high):
        # If current CGPA is greater than pivot (for descending order)
        if students[j].cgpa > pivot:
            i += 1
            students[i], students[j] = students[j], students[i]

    # Place pivot in correct position
    students[i + 1], students[high] = students[high], students[i + 1]
    return i + 1

def merge_sort(students: List[Student]) -> List[Student]:
    """
    Merge Sort algorithm to sort students by CGPA in descending order
    Time Complexity: O(n log n)
    Space Complexity: O(n)

    Args:
        students: List of Student objects
```

```
def generate_large_dataset(size: int) -> List[Student]:
    """
    Generates a large dataset for performance testing

    Args:
        size: Number of student records to generate

    Returns:
        List of Student objects
    """
    first_names = ["Rahul", "Priya", "Amit", "Sneha", "Vikram", "Anjali", "Rohan",
                  "Kavya", "Arjun", "Pooja", "Karan", "Divya", "Sanjay", "Neha", "Aditya"]
    last_names = ["Sharma", "Patel", "Kumar", "Gupta", "Singh", "Reddy", "Iyer",
                 "Nair", "Desai", "Mehta", "Joshi", "Shah", "Verma", "Kapoor", "Rao"]

    students = []
    for i in range(size):
        name = f"{random.choice(first_names)} {random.choice(last_names)}"
        roll_number = f"CS2021{i:06d}"
        cgpa = round(random.uniform(6.0, 10.0), 2)
        students.append(Student(name, roll_number, cgpa))

    return students

def display_top_students(students: List[Student], top_n: int = 10) -> None:
    """
    Displays top N students based on CGPA

    Args:
        students: Sorted list of Student objects (descending by CGPA)
        top_n: Number of top students to display (default: 10)
    """
    print(f"\n{'-'*70}")
    print(f"TOP {top_n} STUDENTS FOR PLACEMENT DRIVE")
    print(f"{'-'*70}")
```

Output:

```
=====
STUDENT PLACEMENT DRIVE - SORTING DEMONSTRATION
=====

1. Creating sample student records...
Total students: 15

2. Sorting using Quick Sort...
Quick Sort completed in 0.000000 seconds

=====
TOP 10 STUDENTS FOR PLACEMENT DRIVE
=====
```

Rank	Roll Number	Name	CGPA
1	CS2021006	Anjali Reddy	9.8
2	CS2021014	Neha Kapoor	9.6
3	CS2021003	Amit Kumar	9.5
4	CS2021012	Divya Shah	9.4
5	CS2021008	Kavya Nair	9.3
6	CS2021001	Rahul Sharma	9.2
7	CS2021005	Vikram Singh	9.1
8	CS2021010	Pooja Mehta	9.0
9	CS2021002	Priya Patel	8.9
10	CS2021009	Arjun Desai	8.8

```
=====
```

Explanation:

The code implements two sorting algorithms, quick sort and merge sort, to sort student records based on their CGPA. It defines a Student class to represent each student's information and includes functions to perform the sorting and display the top students. The runtime performance of both sorting algorithms is measured and displayed, allowing for a comparison of their efficiency. This approach ensures that the code is clean, efficient, and easy to understand while demonstrating the functionality with real-life sample data.

Question 2:

Implementing Bubble Sort with AI Comments

Prompt: Generate a python code for implementing a bubble sort with Comments. Include inline comments explaining key logic like swapping, passes, and termination. Also Provide time complexity analysis of the bubble sort algorithm. Ensure the code is clean and efficient.

Code:

```
import time
import random
from typing import List

def bubble_sort(arr: List[int]) -> List[int]:
    """
    Bubble Sort Algorithm Implementation

    Time Complexity:
    - Best Case: O(n) - when array is already sorted (with optimization)
    - Average Case: O(n2) - typical random array
    - Worst Case: O(n2) - when array is reverse sorted

    Space Complexity: O(1) - sorts in-place, no extra space needed

    Stability: Stable - maintains relative order of equal elements

    Args:
    |   arr: List of integers to be sorted

    Returns:
    |   Sorted list in ascending order
    """
    n = len(arr)

    # Outer loop: Controls the number of passes
    # We need (n-1) passes to guarantee sorting
    for i in range(n - 1):
        # Flag to optimize: detect if array is already sorted
        # If no swaps occur in a pass, array is sorted
        swapped = False

        # Inner loop: Compares adjacent elements
        # Each pass moves the largest unsorted element to its correct position
        # After i passes, last i elements are already in place
        # So we only need to check (n - i - 1) elements
```

```
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            # If current element is greater than next, they are out of order
            if arr[j] > arr[j + 1]:
                # Swap the elements to correct their order
                # This "bubbles" the larger element toward the end
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

                # Mark that a swap occurred
                swapped = True

        # Optimization: If no swaps occurred, array is already sorted
        # Break early to avoid unnecessary passes
        if not swapped:
            break

    return arr
```

```
def bubble_sort_verbose(arr: List[int]) -> List[int]:
    """
    Bubble Sort with detailed step-by-step output for educational purposes

    Args:
    |   arr: List of integers to be sorted

    Returns:
    |   Sorted list in ascending order
    """
    n = len(arr)
    print(f"\nStarting Bubble Sort on array: {arr}")
    print(f"Array length: {n}")
    print("=" * 70)
```

```
# Track total number of comparisons and swaps
total_comparisons = 0
total_swaps = 0

# Outer loop for passes
for i in range(n - 1):
    print(f"\nPass {i + 1}:")
    print(f"  Array before pass: {arr}")

    swapped = False
    comparisons_in_pass = 0
    swaps_in_pass = 0

    # Inner loop for comparisons
    for j in range(0, n - i - 1):
        comparisons_in_pass += 1

        # Comparison step
        print(f"    Comparing arr[{j}]= {arr[j]} with arr[{j+1}]= {arr[j+1]}", end="")

        if arr[j] > arr[j + 1]:
            # Swap step
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            swaps_in_pass += 1
            swapped = True
            print(f"    -> Swap! New: {arr}")
        else:
            print(f"    -> No swap needed")

    total_comparisons += comparisons_in_pass
    total_swaps += swaps_in_pass

    print(f"  Array after pass: {arr}")
    print(f"  Comparisons in this pass: {comparisons_in_pass}")
    print(f"  Swaps in this pass: {swaps_in_pass}")
```

```
def demonstrate_bubble_sort():
    """
    Demonstrates bubble sort with various examples
    """
    print("=" * 70)
    print("BUBBLE SORT DEMONSTRATION")
    print("=" * 70)

    # Example 1: Small unsorted array
    print("\n--- Example 1: Small Unsorted Array ---")
    arr1 = [64, 34, 25, 12, 22, 11, 90]
    print(f"Original: {arr1}")
    sorted_arr1 = bubble_sort(arr1.copy())
    print(f"Sorted: {sorted_arr1}")

    # Example 2: Already sorted array
    print("\n--- Example 2: Already Sorted Array (Best Case) ---")
    arr2 = [1, 2, 3, 4, 5, 6, 7]
    print(f"Original: {arr2}")
    sorted_arr2 = bubble_sort(arr2.copy())
    print(f"Sorted: {sorted_arr2}")

    # Example 3: Reverse sorted array
    print("\n--- Example 3: Reverse Sorted Array (Worst Case) ---")
    arr3 = [90, 80, 70, 60, 50, 40, 30]
    print(f"Original: {arr3}")
    sorted_arr3 = bubble_sort(arr3.copy())
    print(f"Sorted: {sorted_arr3}")

    # Example 4: Array with duplicates
    print("\n--- Example 4: Array with Duplicates ---")
    arr4 = [5, 2, 8, 2, 9, 1, 5, 5]
    print(f"Original: {arr4}")
    sorted_arr4 = bubble_sort(arr4.copy())
    print(f"Sorted: {sorted_arr4}")
```

Output:

```
=====
BUBBLE SORT DEMONSTRATION
=====

--- Example 1: Small Unsorted Array ---
Original: [64, 34, 25, 12, 22, 11, 90]
Sorted:   [11, 12, 22, 25, 34, 64, 90]

--- Example 2: Already Sorted Array (Best Case) ---
Original: [1, 2, 3, 4, 5, 6, 7]
Sorted:   [1, 2, 3, 4, 5, 6, 7]

--- Example 3: Reverse Sorted Array (Worst Case) ---
Original: [90, 80, 70, 60, 50, 40, 30]
Sorted:   [30, 40, 50, 60, 70, 80, 90]

--- Example 4: Array with Duplicates ---
Original: [5, 2, 8, 2, 9, 1, 5, 5]
Sorted:   [1, 2, 2, 5, 5, 5, 8, 9]

--- Example 5: Sorting in Descending Order ---
Original: [3, 7, 1, 9, 4, 6, 2]
Sorted:   [9, 7, 6, 4, 3, 2, 1]

--- Example 6: Step-by-Step Verbose Sorting ---

Starting Bubble Sort on array: [5, 2, 8, 1, 9]
Array length: 5
=====

Pass 1:
  Array before pass: [5, 2, 8, 1, 9]
  Comparing arr[0]=5 with arr[1]=2 -> Swap! New: [2, 5, 8, 1, 9]
  Comparing arr[1]=5 with arr[2]=8 -> No swap needed
  Comparing arr[2]=8 with arr[3]=1 -> Swap! New: [2, 5, 1, 8, 9]
```

```
  Array before pass: [2, 5, 1, 8, 9]
  Comparing arr[0]=2 with arr[1]=5 -> No swap needed
  Comparing arr[1]=5 with arr[2]=1 -> Swap! New: [2, 1, 5, 8, 9]
  Comparing arr[2]=5 with arr[3]=8 -> No swap needed
  Array after pass: [2, 1, 5, 8, 9]
  Comparisons in this pass: 3
  Swaps in this pass: 1

Pass 2:
  Array before pass: [2, 1, 5, 8, 9]
  Comparing arr[0]=2 with arr[1]=1 -> Swap! New: [1, 2, 5, 8, 9]
  Comparing arr[1]=2 with arr[2]=5 -> No swap needed
  Array after pass: [1, 2, 5, 8, 9]
  Comparisons in this pass: 2
  Swaps in this pass: 1

Pass 3:
  Array before pass: [1, 2, 5, 8, 9]
  Comparing arr[0]=1 with arr[1]=2 -> No swap needed
  Array after pass: [1, 2, 5, 8, 9]
  Comparisons in this pass: 1
  Swaps in this pass: 0

Pass 4:
  Array before pass: [1, 2, 5, 8, 9]
  Comparing arr[0]=1 with arr[1]=2 -> No swap needed
  Array after pass: [1, 2, 5, 8, 9]
  Comparisons in this pass: 1
  Swaps in this pass: 0

Array is sorted! Terminating early at pass 4

=====
Sorting Complete!
Total comparisons: 10
Total swaps: 4
Final sorted array: [1, 2, 5, 8, 9]
```

AI Generated Time Complexity Analysis:

- The time complexity of bubble sort is $O(n^2)$ in the worst and average cases, where n is the number of elements in the list. This is because in the worst case, each element needs to be compared with every other element.
- However, in the best case (when the list is already sorted), the time complexity is $O(n)$ because the algorithm will make one pass through the list to check if it is sorted, without performing any swaps. Explanation: The bubble sort algorithm is a simple sorting method that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. It continues to pass through the list until it is sorted. The algorithm is easy to understand and implement, making it a good choice for educational purposes. However, it is not the most efficient sorting algorithm for large datasets due to its $O(n^2)$ time complexity in the worst case. It is best used for small lists or as an introduction to sorting concepts.

Question 3: Quick Sort and Merge Sort Comparison

Prompt: Generate a python code to implement quick sort and merge sort using recursion. Include sample function calls to demonstrate the correctness of the

implementation. Also give explanation of average, best, and worst case complexities. Provide a clean and efficient code with proper comments explaining the logic.

Code:

```
from typing import List

def quick_sort(arr: List[int]) -> List[int]:
    """
    Quick Sort using recursion (returns a new sorted list)
    Average: O(n log n), Best: O(n log n), Worst: O(n^2)
    """
    if len(arr) <= 1:
        return arr # Base case: already sorted

    pivot = arr[len(arr) // 2] # Choose middle element as pivot
    left = [x for x in arr if x < pivot] # Elements less than pivot
    equal = [x for x in arr if x == pivot] # Elements equal to pivot
    right = [x for x in arr if x > pivot] # Elements greater than pivot

    # Recursively sort partitions and concatenate
    return quick_sort(left) + equal + quick_sort(right)

def merge_sort(arr: List[int]) -> List[int]:
    """
    Merge Sort using recursion (returns a new sorted list)
    Best/Average/Worst: O(n log n)
    """
    if len(arr) <= 1:
        return arr # Base case: already sorted

    mid = len(arr) // 2
    left = merge_sort(arr[:mid]) # Recursively sort left half
    right = merge_sort(arr[mid:]) # Recursively sort right half

    return merge(left, right) # Merge sorted halves

def merge(left: List[int], right: List[int]) -> List[int]:
    """Merge two sorted lists into one sorted list."""
    merged = []
```

Output:

```
PS D:\AI assistance coding> & 'c:\Users\sanja\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '57524' '--' 'D:\AI assistance coding\assignment12.3.py'
Original: [38, 27, 43, 3, 9, 82, 10]
Quick Sort: [3, 9, 10, 27, 38, 43, 82]
Merge Sort: [3, 9, 10, 27, 38, 43, 82]
PS D:\AI assistance coding>
```

Explanation:

Quick sort has an average and best case time complexity of $O(n \log n)$ because it efficiently divides the array into smaller sub-arrays and sorts them recursively. However, in the worst case, when the smallest or largest element is consistently chosen as the pivot, it can degrade to $O(n^2)$ due to unbalanced partitions. On the other hand, merge sort always divides the array into halves, resulting in a consistent time complexity of $O(n \log n)$ for all cases, making it more predictable but often less efficient than quick sort for small datasets due to additional space requirements.

Question 4: Real-Time Application – Inventory Management System

Prompt: Generate a python code for Real Time Application of Inventory Management System. The inventory system contains many products each with attributes like Product ID, name, price and stock quantity. Implement a Quickly search for a product by ID or name and Sort products by price or quantity for stock analysis. Suggest the most efficient search and sort algorithms for this use case. Justify the choice based on dataset size, update frequency, and performance requirements. The output should be a table mapping operations - recommended algorithm - justification. Provide a clean and efficient code with proper comments explaining the logic.

Code:

```
import time
import random
from typing import List, Dict, Tuple, Optional
from dataclasses import dataclass
from enum import Enum

# ===== DATA STRUCTURES =====

@dataclass
class Product:
    """
    Represents a product in the inventory system

    Attributes:
        product_id: Unique identifier for the product
        name: Product name
        price: Product price in currency
        stock_quantity: Current stock quantity
    """
    product_id: str
    name: str
    price: float
    stock_quantity: int

    def __repr__(self):
        return f"Product(ID={self.product_id}, Name={self.name}, Price=${self.price:.2f}, Stock={self.stock_quantity})"

    def __str__(self):
        return f"{self.product_id:<10} | {self.name:<20} | ${self.price:>8.2f} | {self.stock_quantity:>6} units"

class SortOrder(Enum):
    """Enumeration for sort order"""
    ASCENDING = "ascending"
    DESCENDING = "descending"

# ===== SEARCH OPERATIONS =====

def search_by_id(self, product_id: str) -> Optional[Product]:
    """
    Search product by ID using Hash Table
    Time Complexity: O(1) average case
    Space Complexity: O(n) for hash table

    Algorithm: Hash Table Lookup
    Justification:
    - O(1) average time complexity for exact match
    - Real-time applications require instant lookups
    - Perfect for unique identifiers
    - Minimal overhead for insertion/deletion

    Args:
        product_id: Product ID to search

    Returns:
        Product object if found, None otherwise
    """
    start_time = time.time()
    product = self.id_index.get(product_id)
    elapsed_time = time.time() - start_time

    if product:
        print(f"Found by ID in {elapsed_time*1000:.4f}ms: {product}")
    else:
        print(f"Product ID '{product_id}' not found")

    return product

def search_by_name(self, name: str) -> List[Product]:
    """
    Search product by name using Linear Search
    Time Complexity: O(n)
    Space Complexity: O(1)
    """
    start_time = time.time()
    products = []
    for product in self.products:
        if product.name == name:
            products.append(product)
    elapsed_time = time.time() - start_time

    print(f"Found by Name in {elapsed_time*1000:.4f}ms: {products}")
    return products
```

```
# ===== SORT OPERATIONS =====

def sort_by_price(self, order: SortOrder = SortOrder.ASCENDING) -> List[Product]:
    """
    Sort products by price using Quick Sort (Random Pivot)
    Time Complexity:  $O(n \log n)$  average case
    Worst Case:  $O(n^2)$  (rare with random pivot)
    Space Complexity:  $O(\log n)$  for recursion stack

    Algorithm: Quick Sort
    Justification:
    - Average  $O(n \log n)$  is optimal for comparison-based sorting
    - In-place sorting requires minimal extra space
    - Cache-friendly due to array access patterns
    - Practical choice for most real-time applications
    - Better than Merge Sort (uses extra  $O(n)$  space)

    Args:
        order: Sort order (ASCENDING or DESCENDING)

    Returns:
        Sorted list of products
    """
    start_time = time.time()

    # Create a copy to avoid modifying original
    products_copy = self.products.copy()

    # Quick Sort implementation
    def quick_sort_helper(arr: List[Product], low: int, high: int, key_func) -> None:
        """
        In-place quick sort using random pivot selection
        """
        if low < high:
            # Partition and get pivot index
```

```
# ===== ALGORITHM RECOMMENDATION TABLE =====

def print_algorithm_recommendations():
    """
    Print recommended algorithms for inventory operations
    with justifications based on use case analysis
    """
    print("\n" + "-" * 120)
    print("ALGORITHM RECOMMENDATIONS FOR INVENTORY MANAGEMENT SYSTEM")
    print("-" * 120)

    recommendations = [
        {
            "Operation": "Search by Product ID",
            "Recommended Algorithm": "Hash Table (Dictionary)",
            "Time Complexity": " $O(1)$  average",
            "Justification": "Exact match on unique identifier; requires instant lookup for real-time transactions"
        },
        {
            "Operation": "Search by Product Name",
            "Recommended Algorithm": "Hash Table (Dictionary)",
            "Time Complexity": " $O(1) + O(k)$ ",
            "Justification": "Multiple products per name possible;  $O(1)$  index lookup +  $O(k)$  to retrieve results"
        },
        {
            "Operation": "Search by Partial Name",
            "Recommended Algorithm": "Linear Search / Trie Index",
            "Time Complexity": " $O(n)$  or  $O(m)$  with Trie",
            "Justification": "Less frequent operation; Trie beneficial for very large catalogs (>100k products)"
        },
        {
            "Operation": "Sort by Price",
            "Recommended Algorithm": "Quick Sort",
            "Time Complexity": " $O(n \log n)$  average",
            "Justification": "Average case optimal; in-place; cache-friendly; minimal extra space vs Merge Sort"
        }
    ]
```

Output:

ALGORITHM RECOMMENDATIONS FOR INVENTORY MANAGEMENT SYSTEM			
Operation	Algorithm	Complexity	Justification
Search by Product ID	Hash Table (Dictionary)	$O(1)$ average	Exact match on unique identifier; requires instant lookup for real-time transactions
Search by Product Name	Hash Table (Dictionary)	$O(1) + O(k)$	Multiple products per name possible; $O(1)$ index lookup + $O(k)$ to retrieve results
Search by Partial Name	Linear Search / Trie Index	$O(n)$ or $O(m)$ with Trie	Less frequent operation; Trie beneficial for very large catalogs (>100K products)
Sort by Price	Quick Sort	$O(n \log n)$ average	Average case optimal; in-place; cache-friendly; minimal extra space vs Merge Sort
Sort by Quantity	Quick Sort	$O(n \log n)$ average	Stock analysis requires frequent sorting; Quick Sort balances speed and space efficiency
Low Stock Alerts	Linear Scan + Quick Sort	$O(n) + O(k \log k)$	Single pass required; then sort k results ($k \ll n$); efficient for real-time monitoring
Range Search (Price/Quantity)	Binary Search Tree / B-Tree	$O(\log n) + O(k)$	For range queries; better than hash table; prefix tree structure enables range operations
Bulk Insert/Update	Batch Processing + Hash Table	$O(m)$ where m = number of updates	Leverage hash table's $O(1)$ insertions; batch updates reduce I/O overhead

Explanation: This code creates a smart inventory system that stores products with details like ID, name, price, and stock quantity. It uses four different organized lists (like filing systems) to find products and sort them by different criteria - one list for quick ID lookups, one sorted by name, one by price, and one by stock quantity. When you search for a product or need a sorted list, the system instantly grabs the right list instead of searching through everything, making it super fast. This approach trades a bit of extra storage space for lightning-fast search and sort operations, which is perfect for real-time businesses that need instant results.

Question 5: Real-Time Stock Data Sorting & Searching

Prompt: Generate a python code for Real Time Stock Data Sorting and Searching. The implemntation should quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently. Implement a sorting algorithm to rank stocks by percentage change. Implement a search function that retrieves stock data instantly when a stock symbol is entered. Optimize sorting with Heap sort and seacrching with Hash Maps. Give a efficient and clean code.

Code:

```
import time
import random
from typing import List, Dict, Tuple, Optional
from dataclasses import dataclass
from enum import Enum

# ===== DATA STRUCTURES =====

@dataclass
class Product:
    """
    Represents a product in the inventory system

    Attributes:
        product_id: Unique identifier for the product
        name: Product name
        price: Product price in currency
        stock_quantity: Current stock quantity
    """
    product_id: str
    name: str
    price: float
    stock_quantity: int

    def __repr__(self):
        return f'Product(ID={self.product_id}, Name={self.name}, Price=${self.price:.2f}, Stock={self.stock_quantity})'

    def __str__(self):
        return f'{self.product_id}<10} | {self.name:<20} | ${self.price:>8.2f} | {self.stock_quantity:>6} units'

class SortOrder(Enum):
    """Enumeration for sort order"""
    ASCENDING = "ascending"
    DESCENDING = "descending"
```

```
# ===== SEARCH OPERATIONS =====

def search_by_id(self, product_id: str) -> Optional[Product]:
    """
    Search product by ID using Hash Table
    Time Complexity: O(1) average case
    Space Complexity: O(n) for hash table

    Algorithm: Hash Table Lookup
    Justification:
    - O(1) average time complexity for exact match
    - Real-time applications require instant lookups
    - Perfect for unique identifiers
    - Minimal overhead for insertion/deletion

    Args:
        product_id: Product ID to search

    Returns:
        Product object if found, None otherwise
    """
    start_time = time.time()
    product = self.id_index.get(product_id)
    elapsed_time = time.time() - start_time

    if product:
        print(f"\n🔍 Found by ID in {elapsed_time*1000:.4f}ms: {product}")
    else:
        print(f"\n❌ Product ID '{product_id}' not found")

    return product

def search_by_name(self, name: str) -> List[Product]:
    """
```

```
# ===== SORT OPERATIONS =====

def sort_by_price(self, order: SortOrder = SortOrder.ASCENDING) -> List[Product]:
    """
    Sort products by price using Quick Sort (Random Pivot)
    Time Complexity: O(n log n) average case
    Worst Case: O(n^2) (rare with random pivot)
    Space Complexity: O(log n) for recursion stack

    Algorithm: Quick Sort
    Justification:
    - Average O(n log n) is optimal for comparison-based sorting
    - In-place sorting requires minimal extra space
    - Cache-friendly due to array access patterns
    - Practical choice for most real-time applications
    - Better than Merge Sort (uses extra O(n) space)

    Args:
        order: Sort order (ASCENDING or DESCENDING)

    Returns:
        Sorted list of products
    """
    start_time = time.time()

    # Create a copy to avoid modifying original
    products_copy = self.products.copy()

    # Quick Sort Implementation
    def quick_sort_helper(arr: List[Product], low: int, high: int, key_func) -> None:
        """
        In-place quick sort using random pivot selection
        """
        if low < high:
            # Partition and sort pivot index
```

```
# ===== ALGORITHM RECOMMENDATION TABLE =====

def print_algorithm_recommendations():
    """
    Print recommended algorithms for inventory operations
    with justifications based on use case analysis
    """
    print("\n" + "=" * 120)
    print("ALGORITHM RECOMMENDATIONS FOR INVENTORY MANAGEMENT SYSTEM")
    print("=" * 120)

    recommendations = [
        {
            "Operation": "Search by Product ID",
            "Recommended Algorithm": "Hash Table (Dictionary)",
            "Time Complexity": "O(1) average",
            "Justification": "Exact match on unique identifier; requires instant lookup for real-time transactions"
        },
        {
            "Operation": "Search by Product Name",
            "Recommended Algorithm": "Hash Table (Dictionary)",
            "Time Complexity": "O(1) + O(k)",
            "Justification": "Multiple products per name possible; O(1) index lookup + O(k) to retrieve results"
        },
        {
            "Operation": "Search by Partial Name",
            "Recommended Algorithm": "Linear Search / Trie Index",
            "Time Complexity": "O(n) or O(n) with Trie",
            "Justification": "Less frequent operation; Trie beneficial for very large catalogs (>100K products)"
        },
        {
            "Operation": "Sort by Price",
            "Recommended Algorithm": "Quick Sort",
            "Time Complexity": "O(n log n) average",
            "Justification": "Average case optimal; in-place; cache-friendly; minimal extra space vs Merge Sort"
        }
    ]
```

```
import time
import random
import heapq
from typing import List, Dict, Optional, Tuple
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from enum import Enum

# ===== DATA STRUCTURES =====

class SortOrder(Enum):
    """Enumeration for sort order"""
    ASCENDING = "ascending"
    DESCENDING = "descending"

@dataclass
class StockData:
    """
    Represents real-time stock data

    Attributes:
        symbol: Stock ticker symbol (e.g., 'AAPL', 'GOOGL')
        company_name: Full company name
        current_price: Current stock price
        previous_close: Previous closing price
        daily_change: Absolute price change (current - previous)
        percentage_change: Percentage change ((change / previous) * 100)
        volume: Trading volume
        market_cap: Market capitalization in billions
        timestamp: Last update timestamp
    """
    symbol: str
    company_name: str
    current_price: float
    previous_close: float
    daily_change: float

# ===== REAL-TIME STOCK DATA SYSTEM =====

class RealTimeStockSystem:
    """
    Real-time stock data management system with optimized search and sort operations
    """

    def __init__(self):
        """Initialize the stock system with optimized data structures"""
        self.stocks: List[StockData] = [] # Main stock list
        self.symbol_index: Dict[str, StockData] = {} # Hash map for O(1) symbol search
        self.sector_index: Dict[str, List[StockData]] = {} # Hash map for sector grouping
        self.last_update_time = datetime.now()

    def add_stock(self, symbol: str, company_name: str, current_price: float,
                  previous_close: float, volume: int, market_cap: float,
                  sector: str = "General") -> None:
        """
        Add a stock to the system
        Time Complexity: O(1) average case for hash table insertion

        Args:
            symbol: Stock ticker symbol
            company_name: Full company name
            current_price: Current stock price
            previous_close: Previous closing price
            volume: Trading volume
            market_cap: Market capitalization in billions
            sector: Stock sector/category
        """
        # Calculate daily change metrics
        daily_change = current_price - previous_close
        percentage_change = (daily_change / previous_close * 100) if previous_close != 0 else 0

        # Create stock object
        stock = StockData(
            symbol=symbol,
```

Output:

NFLX205	Company 2056	\$ 250.84	+367.44%	40,700,297
AAPL926	Company 9260	\$ 250.07	+367.47%	51,032,283
GOOGL65	Company 651	\$ 458.89	+367.65%	12,243,252
FB574	Company 5745	\$ 450.60	+368.18%	70,827,906
INTC141	Company 1419	\$ 485.05	+368.22%	72,537,439
INTC505	Company 5059	\$ 261.36	+368.32%	65,564,446
AMD440	Company 4408	\$ 355.55	+368.72%	67,251,345
GOOGL737	Company 7371	\$ 357.32	+368.76%	10,896,589
AMZN363	Company 3633	\$ 393.82	+368.99%	63,937,567
AMD226	Company 2268	\$ 420.39	+369.03%	95,007,128
TSLA485	Company 4854	\$ 395.52	+369.05%	55,656,195
NFLX679	Company 6796	\$ 403.40	+369.32%	5,497,053
INTC727	Company 7279	\$ 318.28	+369.89%	82,733,124
AMD431	Company 4318	\$ 198.01	+369.98%	4,348,613
AAPL684	Company 6840	\$ 424.69	+370.18%	19,867,784
GOOGL201	Company 2011	\$ 343.91	+371.05%	84,365,409
INTC353	Company 3539	\$ 352.13	+371.29%	59,282,535
INTC216	Company 2169	\$ 210.62	+371.76%	17,636,793
NFLX109	Company 1096	\$ 315.84	+373.07%	63,109,821
NVDA157	Company 1577	\$ 468.53	+373.30%	96,059,178
AAPL287	Company 2870	\$ 428.48	+373.50%	91,492,550
NFLX952	Company 9526	\$ 296.56	+373.56%	82,661,265
NFLX668	Company 6686	\$ 376.24	+373.91%	23,982,284
NFLX663	Company 6636	\$ 380.44	+373.93%	43,207,190
FB688	Company 6885	\$ 491.93	+374.45%	18,430,842
NVDA313	Company 3137	\$ 106.44	+374.57%	2,883,113
NFLX530	Company 5306	\$ 203.88	+374.68%	99,060,602
MSFT633	Company 6332	\$ 453.24	+374.69%	68,711,527
INTC653	Company 6539	\$ 436.83	+374.72%	40,794,301
AMD548	Company 5488	\$ 292.23	+375.05%	1,036,158
AMD937	Company 9378	\$ 229.58	+375.11%	23,818,543
AMD21	Company 218	\$ 492.95	+376.19%	76,739,507
TSLA999	Company 9994	\$ 319.35	+376.81%	43,107,472
GOOGL842	Company 8421	\$ 471.52	+377.18%	34,697,200
FB956	Company 9565	\$ 485.84	+377.32%	76,239,411
FB482	Company 4825	\$ 210.63	+377.43%	26,544,150

Explanation: This code creates a lightning-fast stock trading system that finds any stock by its symbol in less than 1 microsecond using a Hash Map (like a super-fast phone book). When you need to see which stocks gained or lost the most money, it uses Heap Sort to organize them quickly and reliably without getting slow. For quick queries like show me top 10 gainers, it uses a smart Heap Selection technique that's much faster than sorting everything perfect for trading alerts. The system can handle thousands of stocks while keeping search time under 1 millisecond and sort time under 5 milliseconds, making it perfect for real-time trading platforms where speed matters.